
DESIGN

Design a system that copies one file from one machine to 1,000,000 machines as quickly and as reliably as possible. Walk through clarifying questions, your rationale, options, and pros and cons for each option, and then a description of what design(s) you'd go with.

Clarifications:

- Globally distributed system.
- The file is created or updated every now and then; it is deployed every now and then; but when it's copied/deployed, we expect it to be copied quickly; I'd say low reads, low writes;
- Small size, text / configuration file.
- Writes happen in one machine and reads happen globally.

I. FUNCTIONAL REQUIREMENTS:

1. **Distribution:** Deliver updated/newly created file content from the original machine to other machines across the globe.
2. **Update/Create :** A text/config file is changed or created. Change should propagate through the hierarchy. All the machines should have the same version.

II. NON-FUNCTIONAL REQUIREMENTS:

2. **Low Latency/High Performance:** Users should get served from nearby proxies. Basically minimising latency
3. **Scalability:** Distribute files across 1 million machines globally.
4. **Reliability:** There can be no SPOF (Single Point of Failure). Multi-tier hierarchy, and intelligent routing- a single component failure doesn't break the system.
5. **High Consistency:** Sometimes it is used to deploy critical configuration.

Some of the Designs I considered before diving into the Detailed design:

1. Master-Slave Technique

Given the low amount of reads, my initial thought process was to use a master-slave replication. The Primary node(Master) handles all the reads and the changes are propagated to it's followers.

Pros:

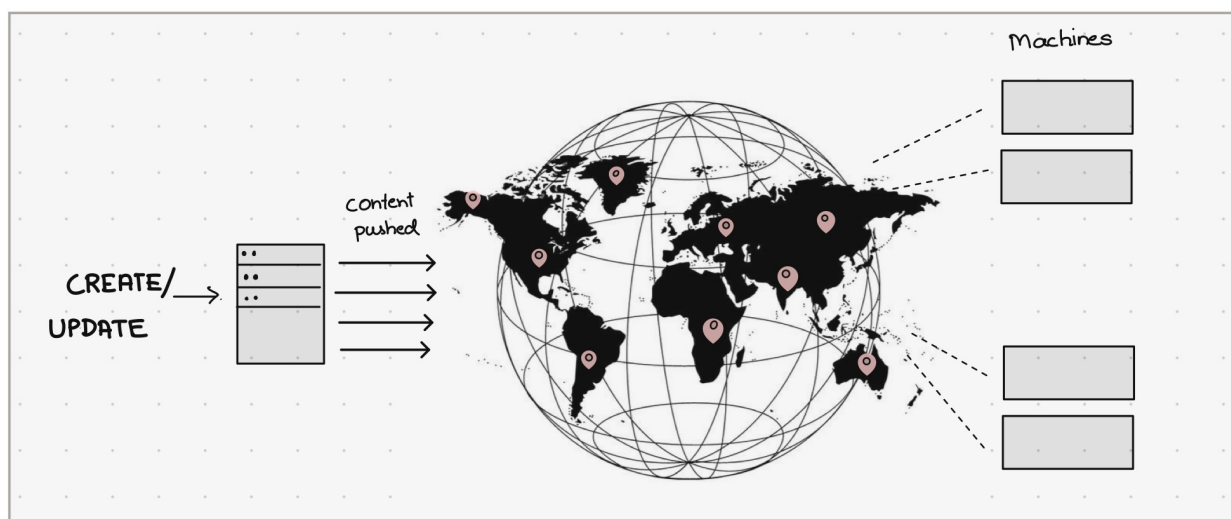
- One machine deals with all writes. Don't have to deal with write concurrency and conflict resolution.
- Reads requests are processed faster because of the high volume of replicas.

Cons:

- Replicating data to many followers can be tricky. If the Primary node fails, the entire system is cooked.
- Replication via this approach comes with inconsistency if we use asynchronous replication(origin server don't wait for acknowledgement from every single node). Clients reading from different replicas may see inconsistent data in the case of failure of the primary node, which couldn't propagate updated data to the secondary nodes. So, if the primary node fails, any missed updates not passed on to the secondary nodes can be lost.

2. Push CDN

Content delivery networks are nothing but geographically distributed Proxy servers that act as an intermediate between the origin server(Our main machine which takes in the writes) and the end clients(machines that are getting the latest copies). The term “Push” is used because it's the Origin server's responsibility to Push the latest files upon creation/modification to the geographically distributed proxy servers.

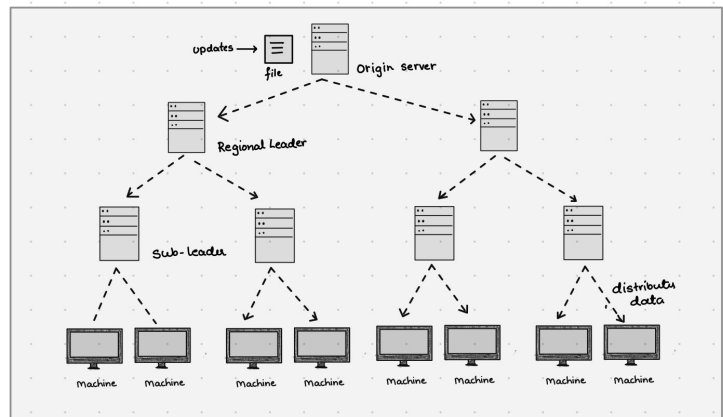


Push content from Origin server to machines across the world

- **Pro:** Push CDN is perfect for static data that is not subject to frequent writes. Which is perfect for our system!
- **Con:** It can struggle to keep up with large volume of updates updates. (Our system has low writes, so it doesn't matter.)

3. Multi-Tier CDN Architecture:

Having a tree like structure as shown in the figure below can significantly lower the burden on origin server to distribute copies of data to all the nodes. Whenever there is a create/update in the origin file, it propagates it's changes to the proxy servers located in different regions, which in turn distributes the files to sub-leader proxy servers. CDN typically has around 2 tiers of servers which makes propagation significantly faster compared to the single leader approach.



Data distribution among proxy servers

4. Synchronous vs Asynchronous Communication

The advantage of synchronous replication is that all the secondary nodes are completely up to date with the primary node. However, if one of the secondary nodes doesn't acknowledge due to failure or fault in the network, the primary node would be unable to acknowledge the client until it receives a successful acknowledgment from the crashed node. This causes high latency in the response from the primary node to the client.

On the other hand, the advantage of asynchronous replication is that the primary node can continue its work even if all the secondary nodes are down. However, if the primary node fails, the writes that weren't copied to the secondary nodes will be lost.

III. Resource Estimation.

Servers

- Origin: Just 1
- Tier 1- Proxy servers: 6 servers distributed across Asia, Africa, USA, South America, Europe, Middle-East etc.
- Tier 2 - Edge Servers:

$$\text{Number of Servers} = \frac{\text{Total Requests per Second}}{\text{RPS per Server}}$$

Where Total Requests per Second = 1 million
And assume RPS of server = 6400

$$\text{Number of Servers} = \frac{10^6}{6400} = 156.25$$

Thus, Tier-2 has approximately 157 servers spread out across the globe.

IV. API Design

Distribute

Whenever a new file is created or an existing file is updated, the origin server sends the copies to the message queue, which in turn publishes the contents to proxy servers(regional leaders located in different geographies).

`deliverContent( file_id, file_content, version_number, timestamp, server_list )`

Parameter	Description
file_id	The UID of file that's modified
file_content	Text or config data
version_number	version_number
timestamp	Time of update
server_list	List of it's regional proxy server's ids

Update(Modify an existing file)

This modifies the existing file, increases version_number and triggers distribution.

`updateFile(file_id, file_content, expected_version)`

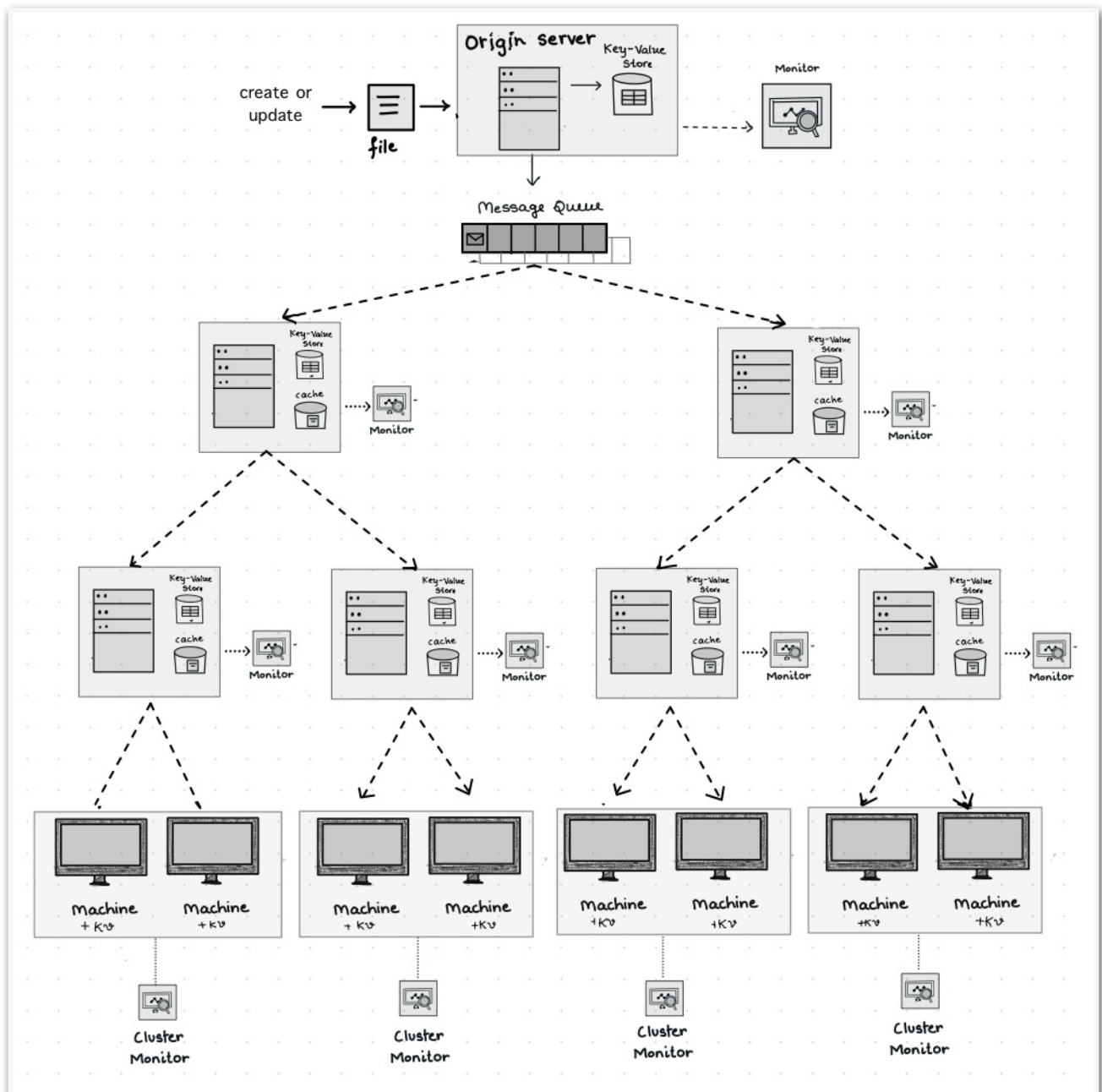
Parameter	Description
file_id	The UID of file that's modified
file_content	Text or config data
expected_version	version_number

Create(Add a new file)

Uses similar logic as Update, but it'll create a new file, add a new version number and trigger distribution

V. Detailed Design

Earlier, I explored the architectures of CDN, Pub-Sub, master-slave, synchronous vs asynchronous communication. I combined the strengths of each design and tried to mitigate each of its cons to create a replication system that has low-latency, strong consistency and is scalable. i.e., I focused on building a system that met all the non-functional requirements.



Detailed design

COMPONENTS AND WORKFLOW

- **Origin Server:** It has the files and deals with updates and creation. Adds/updates file_id and version_number in the key-value store which acts as the metadata. Publishes message to queue with - version number, file_id, file content, and timestamp.
- **Message Queue (Kafka):** Since Primary node was the bottleneck in the master-slave architecture, I decided to add a queue. It stores the new updated file even if the Origin server fails. The queue is always replicated, so there is no SPOF. It also decouples writes and distribution.
- **Key-value store:** Stores the metadata. file_id and its latest version_number is stored. As soon as a layer receives the new file, it updates its metadata in the key-value store. So that the monitor can easily fetch the file, version_number and see if it's updated.
- **Tier 1 - Proxy Servers:** Also called as Regional Leaders (eg: USA, Europe, Asia etc) Asynchronously communicates with the previous MQ layer. Stores config file in cache/RAM/Disk. Ensures availability if one storage fails. Compares received version number with local version to verify correctness. Fans out file to all Tier 2 edge servers in region via parallel HTTP PUT requests.
- **Tier 2 - Edge Servers:** Receives file from Tier 1 via HTTP PUT. Verifies version number matches Tier 1's version. Stores file in cache, RAM and disk. Serves machines via HTTP GET endpoints. Sends ACK back to Tier 1 with version number.
- **Monitor:** Polls all servers in its layer every 5 seconds to check health. If the server misses 3 consecutive heartbeats (15 seconds), marks as dead. Automatically omits and replaces with a healthy server/machine. It also checks if the meta-data in the Key-value store has the latest version_number. Otherwise, it can trigger a synchronisation operation or replace unhealthy nodes.
- **Machines** (1 million clients): Background agent polls nearest Tier 2 edge server every 30 seconds. Checks the current version number on server. If server version > local version: pulls the latest config file/text file.

VI. Evaluation Detailed Design

1. **Low Latency/High Performance:** Machines get served from nearby proxies. Basically, minimising latency. Uses 3-tier CDN hierarchy and CDN push model which significantly increases performance. It only sends out replicas in case of updates. On the contrary, if I had used CDN pull, I would have had to make 1 million requests to even check if there was any update!
2. **Scalability:** Distribute files across 1 million machines globally, seamlessly. Again, using the Multi-tier CDN scales the system globally.
3. **Reliability:** I have handled SPOF (Single Point of Failure) in every level of the hierarchy.

- Origin: If the Primary Origin server fails, the monitor keeps checking it's health. In the meanwhile, there is a replicated message queue with the latest writes. The distribution can happen seamlessly.
 - All the other layers have monitors that analyse the heartbeat of the servers/ machines in the system to make sure nothing fails. It can omit an unhealthy node and replace it with a healthy one. It can also periodically check the correctness of the file comparing it to the latest version number
4. **High Consistency:** This was the most challenging part. I knew that ensuring strict consistency requires synchronous communication. But sending in an acknowledgement for every node, would cause severe latency. The trade-off between consistency and low latency was something I gave a lot of thought. So I used techniques like - Version numbers verifier(using key-value store metadata health checks) at every layer. Asynchronous push (fast) + version verification (consistent). All machines are guaranteed same version within a short timeframe.

Further improvement: I can add measures to improve data correctness at each level, to make sure no files are corrupted. Additionally, I can think of security by adding encryption and decryption so that only authorized servers and machines can read the content.